

7 Dars

Training Performance, Optimization, LLM arxitekturalari (Final) va Tokenization.

1. LLM vs SLM. Statik ehtiyoqlikdan $\rightarrow$ Inchiyan tor doiradagi SLM va Global energiya sarf qilishiga ega ulkan modelga o'g'ir LLM
2. Token vs Word: Söz Inson söz birligi. Token - BPE bölakchasi (subword)
3. Claude vs Local LLM: Bulutli Superkompyuter servislari va xususiy GPUdagi lokal modeller
4. Chinchilla & Scaling Law: D = 20N qoidasi; Layerlar optimal pretraining.
5. Embedding vs Encoding: Statik shakl o'zgarishi
Encoding vs top o'lchamli semantic va olinsanik vektorial koordinatalar.
6. Attention & layers. Q, K, orqali kontekst shakllantiruvchi Attention hands faktual bilimlarni yushtirchi FFN qatnamlari.
7. Parallel vs Serial: Attention va FFNning ketma ket Serial yoki Kernel Fusion o'zgarishi parallel hisoblanishi.
8. Layer Norm vs RMS Norm: Mean (M) va shift (B) ni chiqarib tashlab, GPU xotira o'tkazuvchanligini 10% 50% gacha

Arxitektura codi tayyor bo'lgandan keyin eng masal-
yatli eng qimmat bosqich. Train Phase.

Ter Arzon Xatbichlorisiz

3ta Asosiy Engineering Ustuniga tayjari

LLM Training Performance.



Hyperparameters	Training Stability	Regularization
LR	Exploding Grad	Overfitting
Batch Size	Loss = NaN/Inf	Weight Decay
Optimizer	LR Warm Up	Dropout
Warm Up Steps	Grad Clipping	
Weight Decay		

Training Hyperparameters

Hyperparameters $\rightarrow$ be train jarayanida avval
engineer tomonidan qoldir solinadigan
model train terligi va sifatini belgi loadi
boshqaruv dastagidlar.

LR $\rightarrow$ Har bir optimizer loss gradient ga qarab
Neuron varularni neolog lig katta mantigil
qadam bilan o'zgartirishi belgi loadi co'vatozi
LR katta bolsa model konvergensiya qi lmaydi
LR o'ta kichik bolsa model o'zgarish; sig'lar
yillomga cho'xilib ketadi.

Batch size $\rightarrow$ Parallell ravishda GPU ga bir vaqtda yuklanadigan tokenlar va matnlar to'plami
Katta batch size GPU parallelizatsiya max foydalanish imkonini beradi.

Optimizer (AdamW)

Gradientlarni hisoblab, Neyron vaarlarni yangilovchi matematik algoritim. U holda birinchi va ikkinchi tartibli momentlarni (m va v) hisoblovchi AdamW.

Warm up Steps

Training eng boshida LR o'dan belgi ~~larasiga~~ normal qiymatgacha sekin asta va ehtiyofta bilan oshirib boradigan dastlabki iteratsiya soni (masalan dastlabki 2000 qadam)

Weight Decay $\rightarrow$ Neyron weightlarini (W) shiddat bilan o'sib ketishini o'lchov shuvchi va ularning nolga qarab tortuvchi L2 penalt mexanizmi.

Training Stability

LLM pre-trained jarayoni yuzlab million dollar va oylar vaqt talab qiladi. Shu sababli train jarayonining barqarorligi (stability) eng birinchi daryali masalalar.

Problem

Exploding Gradients

- Kōp qavatli (100+ layers)
- Warm up qilimaganida
- Loss / noisy data

Symptom

Loss birdaniga ∞ o'tlaydi
Loss = NaN yoki Loss = inf

Solution $\rightarrow$ LR Warmup ($0 \rightarrow$ Peak LR avaziga)
Gradient Clipping (Norm threshold = 1.0)

Result

Loss Boshda bosqichlarda tez
Barqaror va sōnib boradi.

Exploding Gradients.

1. Layers 100+
2. Boshlangichda loss ehtiyotkorlik bilan beriladi
Warm up qilimaganida.
3. Train data buzilgan / noisy yoki faliq.

Model butunlay isholan diyor
1. LR $\rightarrow$ Training bilan yuqori LR boshida
dastlabki 2000 qadamgacha LR qiymati
 $0 \rightarrow$ LR max shkalasida diyorli oshirib beriladi
2. Gradient Clipping. Threshold 1.0

Regularization : Oversitting va LLM lardagi
Oltin qoida.

Oversitting : Model Datablogi umumiy mantiq
va qonuniyatlarni o'rgatish o'rniga matnlarni
kõr kõwona yodlab oladi

Regularization : Model yodlab olish o'rniga
matnning ichki semantic mantiqini
tushunishga majbur qiluvchi matematik
usullar to'plamini.

Regularization in LLM: Golden Rule.

1. Weight Decay : Neyron Weight hadblar tez
osib ketish, ushbu penalte uni o'zgarish to'xta-
di va neyronlarni vazmli ushlab turadi.
2. Drop out : Vagtincha ba'zi Neyronlar
õchirish.

LLM da oltin qoida: LLM larni training
qilishda Drop out ≈ 0.0 qo'yiladi.
Chunki trioulab turelar ulkan Datablog
Drop out qollash under training va
qimmatli compute resurslarning isrof
bolishiga olib keladi. Faqat Weight
Decay ishlatilishga yetarli.

LLM Architecture

Eski (original Vaswani 2017)

Yangi (romonaviy SOTA - Llama-3, Mistral)

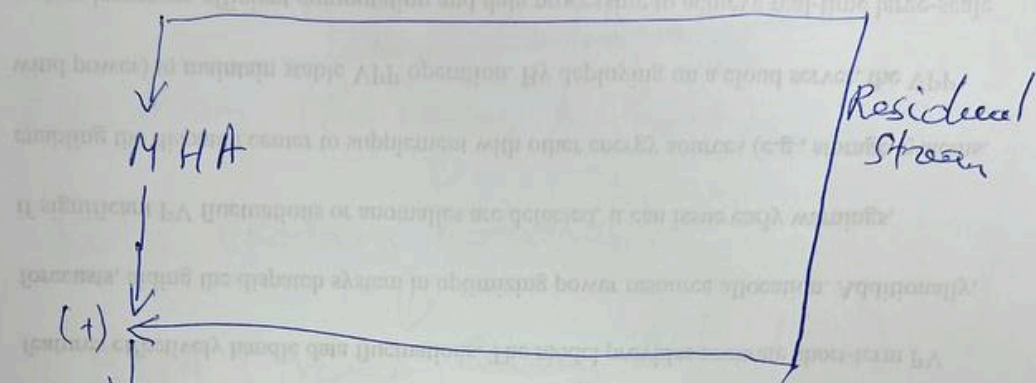
Komponentlar	Eski xolat	Yangi xolat
Arxitektura turi	Encoder-Decoder	Decoder only
Normalizatsiya tartibi	Post-Norm (Amallardan so'ng)	Pre-Norm (Amallardan oldin)
Normalizatsiya Algoritmi	Layer Norm (Mean va Variance)	RMSNorm (10-50 th)
Position Encoding	Absolute Sinusoidal Positional Encoding	RoPE (Rotary Position Embedding Complex)
Activation Function	Relu / gELU	SiLU / GLU High Residual
Attention Variant	MHA (Heavy KV)	GQA (Tejavar)
Blok tuzilishi	Ketma-ket	Serial yoki Kernel Fused Parallel
Drop out	\$0.1 Model boy/ob	\$0.0 \$

Esri. Arxitektura (Post-Norm, Layer Norm,
Relu, MHA)

[Input Tokens]



[Absolute Position Encoding]



Layer Norm (Mean + Variance)



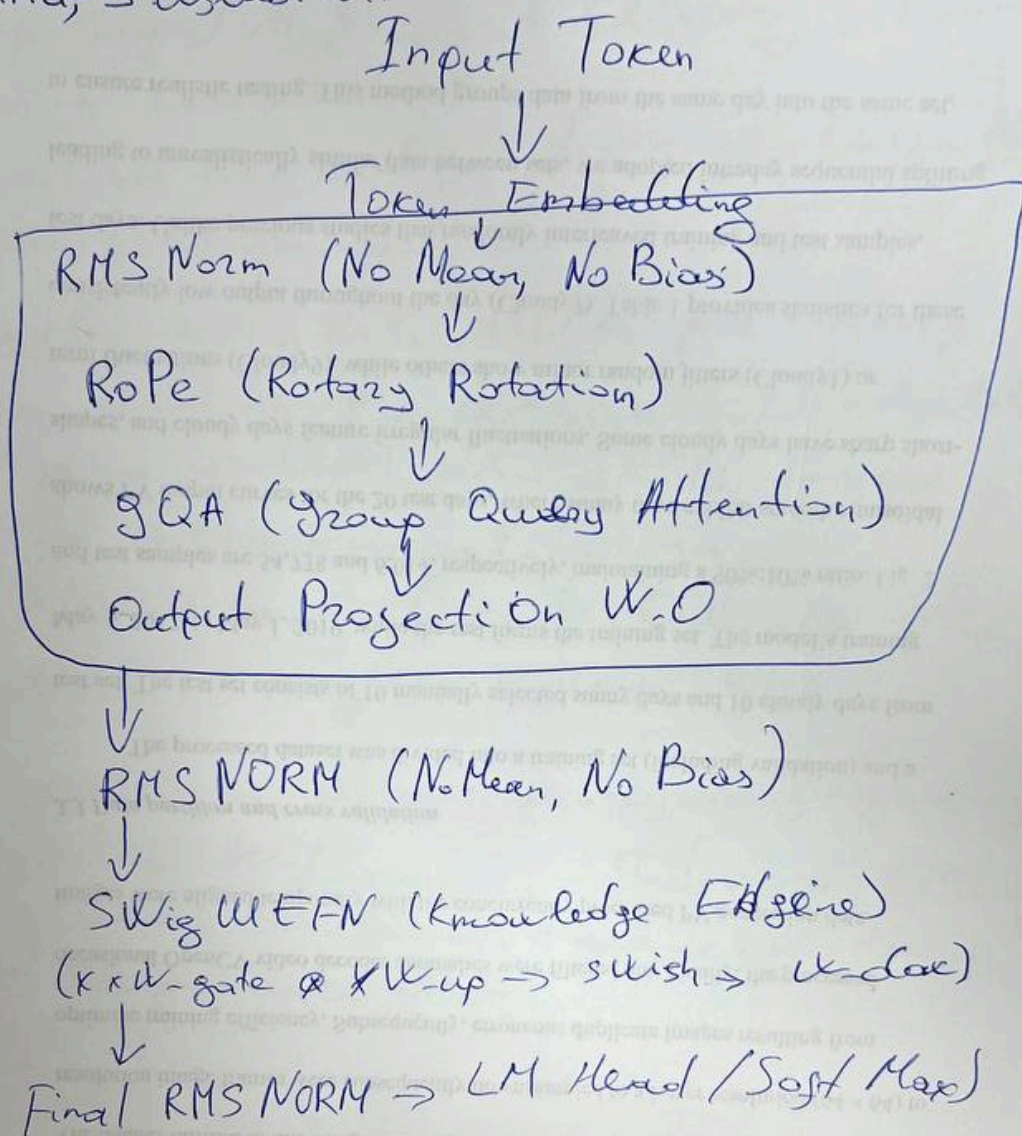
Feed-Forward (Relu)



Layer Norm (Mean + Variance)

Yangi arxitektura.

Pre-Norm, RMS Norm, SWiGLU, RoPE, GQA
Llama, 3 uslubida.



Xulosalar.

1. Training Optimizatsiya: LLM o'g'itishda bangarodilica erishish uchun LR Warm up (2000 steps) va gradient Clipping (Threshold 1.0) kuzatib boriladi.

Gradient Exploding holatida $Loss = NaN$ xatosi kelib chiqadi.

2. Regularization Rule: Ulkan pre-training ma'lumotlarida Dropout = 0.1 qilib o'ldirilgan, ortiqcha yodlashlarni o'ldirish fagat Weight Decay orqali olinadi.

3. Architecture $\rightarrow$ Decoder Only + RoPE - Norm + RMS Norm + Sigmoid + RoPE + GQA.

4. Tokenization Rule: kompyuter fagat raqamli tushunadi. Subword BPE algoritmi so'zlarni teganlar token ID raqamga o'tiradi.

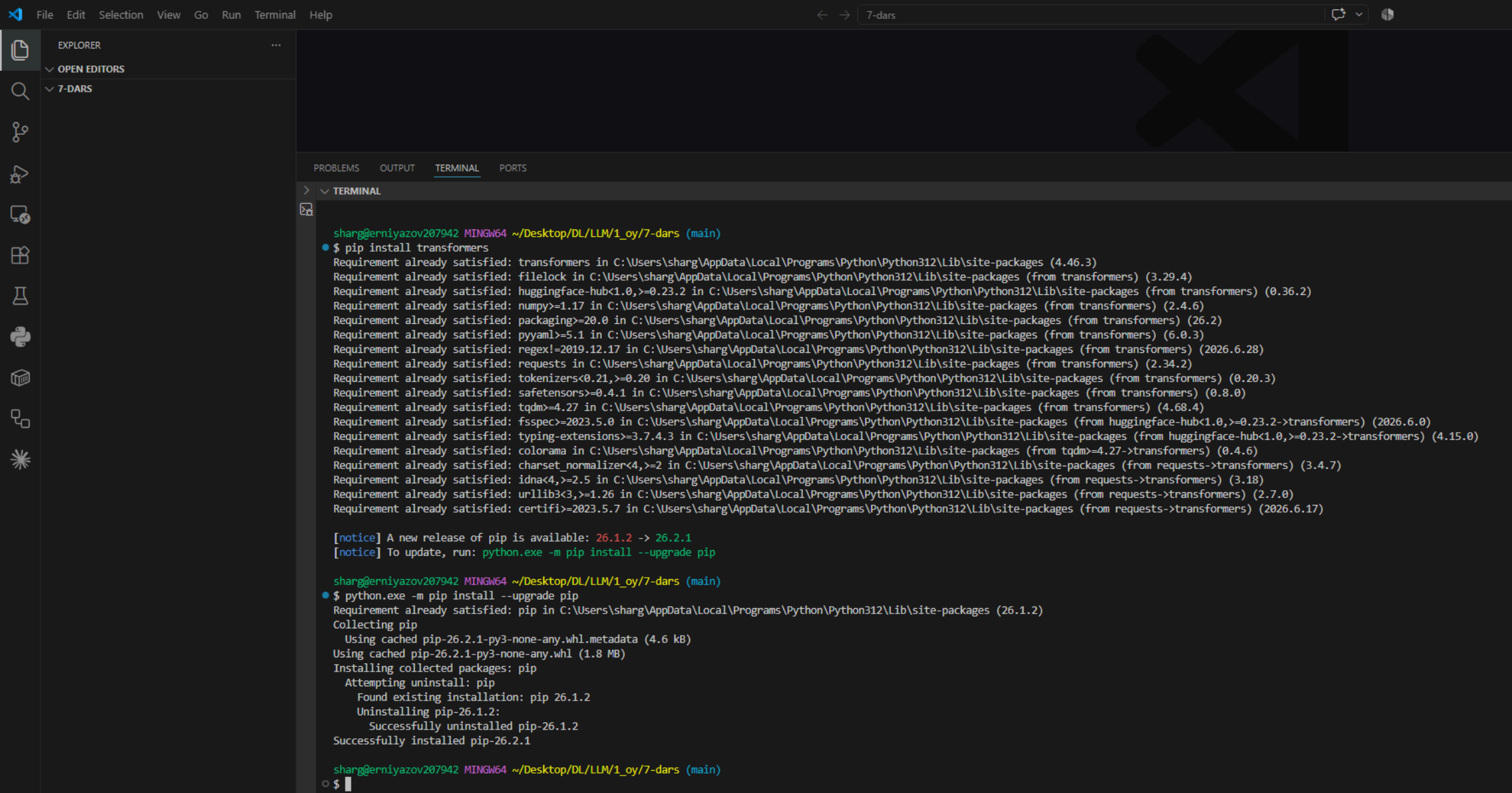
Tocênizer matnni raganga aylatuvchi
kuzat LLM modeling o'cinas.

3 (4 + 0120) bosh joyni bildiruvchi maxsus
belgi GPT-2 ning byte-level BPE usulida
ishlatiladi.

Korta /kidic hq, finish belgi, joylashuv,
barчасi boshqa token beradi.

Inglizcha matn samarali tokenlashu
o'zbekcha matn esa 2 barobar

ko'proq tokenga bo'linib - sababli
Vocab asosan inglizcha ko'pchilik o'zgaradi.



> **✓** **TERMINAL**



```
sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1 oy/7-dars (main)
```

- `$ huggingface-cli login`

⚠ Warning: 'huggingface-cli login' is deprecated. Use 'hf auth login' instead.

A token is already saved on your machine. Run ``hf auth whoami`` to get more information or ``hf auth logout`` if you want to log out. Setting a new token will erase the existing one.

To log in, `huggingface hub` requires a token generated from <https://huggingface.co/settings/tokens>.

Token can be pasted using 'Right-Click'.

Enter your token (input will not be visible):

```
Add token as git credential? (Y/n) Y
```

Token is valid (permission: write).

The token `celltrriage-deploy` has been saved to C:\Users\sharg\.cache\huggingface\stored tokens

```
Your token has been saved in your configured git credential helpers (manager).
```

Your token has been saved to C:\Users\sharg\.cache\huggingface\token

Login successful.

The current active token is: `celltrriage-deploy`

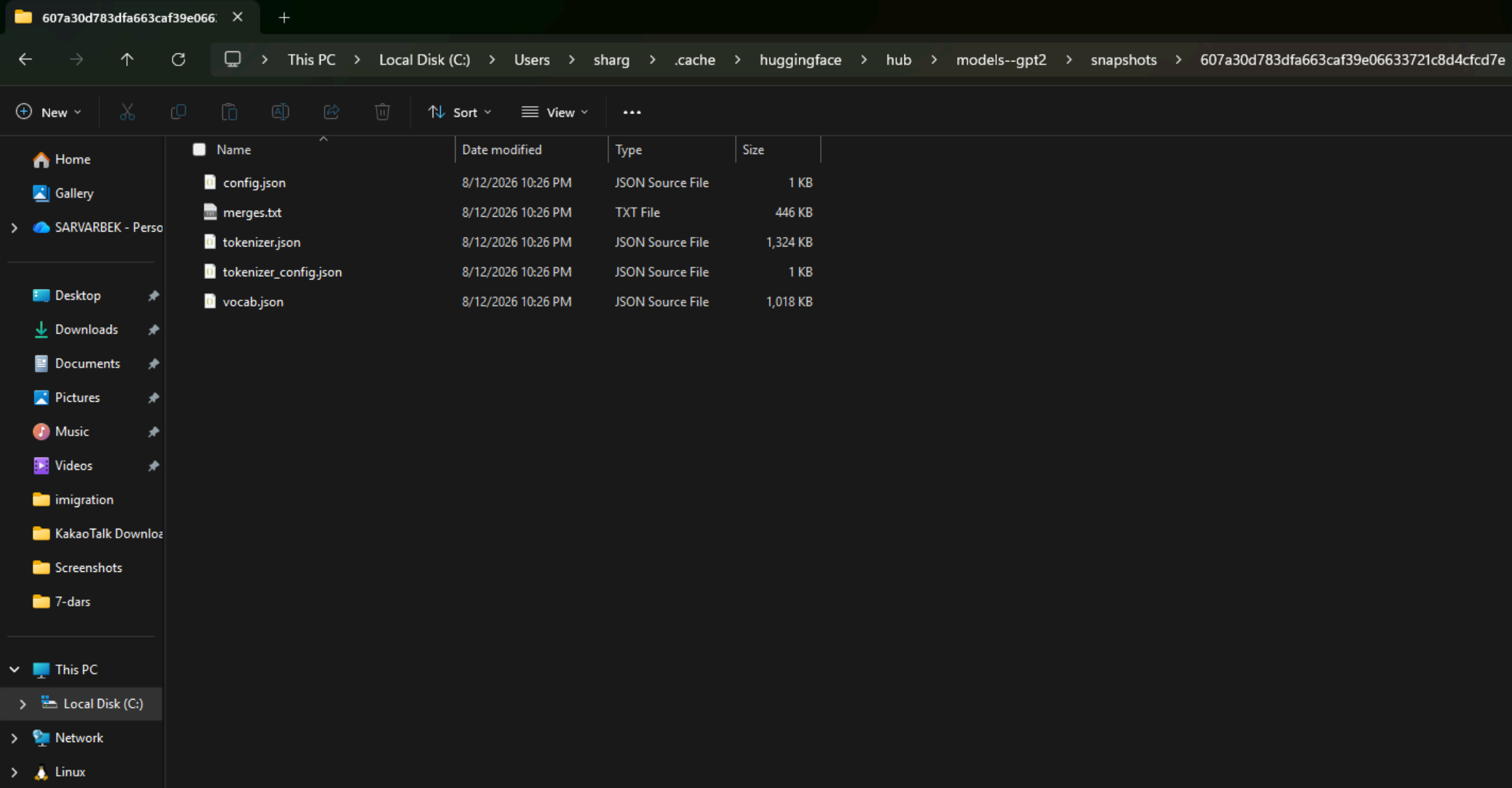
```
sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)
```

- `$ hf auth whoami`

```
user: Sarvarbek13
```

```
sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1 oy/7-dars (main)
```

○ \$



FileEditSelectionViewGoRunTerminalTerminal7-dars

EXPLORER...PROBLEMSOUTPUTTERMINALPORTS

OPEN EDITORS

7-DARS

1.png

2.png

3.png

tokenization.py

TERMINAL

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$ python tokenization.py

['I', 'love', 'AI']

['Machine', 'learning', 'is', 'powerful', '.']

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$ cat >> tokenization.py << 'EOF'

print(tokenizer.tokenize("I LOVE ai"))

EOF

python tokenization.py

['I', 'love', 'AI']

['Machine', 'learning', 'is', 'powerful', '.']

['I', 'LOVE', 'a', 'i']

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$ cat >> tokenization.py << 'EOF'

print(tokenizer.tokenize("Hello, world! How are you?"))

EOF

python tokenization.py

['I', 'love', 'AI']

['Machine', 'learning', 'is', 'powerful', '.']

['I', 'LOVE', 'a', 'i']

['Hello', ',', 'world', '!', 'How', 'are', 'you', '?']

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$ cat >> tokenization.py << 'EOF'

print(tokenizer.tokenize("I have 25 apples and 3 bananas."))

EOF

python tokenization.py

['I', 'love', 'AI']

['Machine', 'learning', 'is', 'powerful', '.']

['I', 'LOVE', 'a', 'i']

['Hello', ',', 'world', '!', 'How', 'are', 'you', '?']

['I', 'have', '25', 'apples', 'and', '3', 'bananas', '.']

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$ cat >> tokenization.py << 'EOF'

print(tokenizer.tokenize("Bugun 7-darsni tugatdik"))

EOF

python tokenization.py

['I', 'love', 'AI']

['Machine', 'learning', 'is', 'powerful', '.']

['I', 'LOVE', 'a', 'i']

['Hello', ',', 'world', '!', 'How', 'are', 'you', '?']

['I', 'have', '25', 'apples', 'and', '3', 'bananas', '.']

['Bug', 'un', '67', '-', 'd', 'ars', 'ni', 'tug', 'at', 'd', 'ik']

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$ [200~cat >> tokenization.py << 'EOF'

> tokens = tokenizer.tokenize("I love AI")

> for t in tokens:

> print(repr(t), "->", repr(tokenizer.convert_tokens_to_string([t])))

> EOF

bash: [200~cat: command not found

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$ cat >> tokenization.py << 'EOF'

tokens = tokenizer.tokenize("I love AI")

for t in tokens:

print(repr(t), "->", repr(tokenizer.convert_tokens_to_string([t])))

EOF

python tokenization.py

['I', 'love', 'AI']

['Machine', 'learning', 'is', 'powerful', '.']

['I', 'LOVE', 'a', 'i']

['Hello', ',', 'world', '!', 'How', 'are', 'you', '?']

['I', 'have', '25', 'apples', 'and', '3', 'bananas', '.']

['Bug', 'un', '67', '-', 'd', 'ars', 'ni', 'tug', 'at', 'd', 'ik']

'I' -> 'I'

'love' -> 'love'

'AI' -> 'AI'

sharg@erniyazov207942 MINGW64 ~/Desktop/DL/LLM/1_oy/7-dars (main)

\$

OUTLINE

TIMELINE

0 0 0